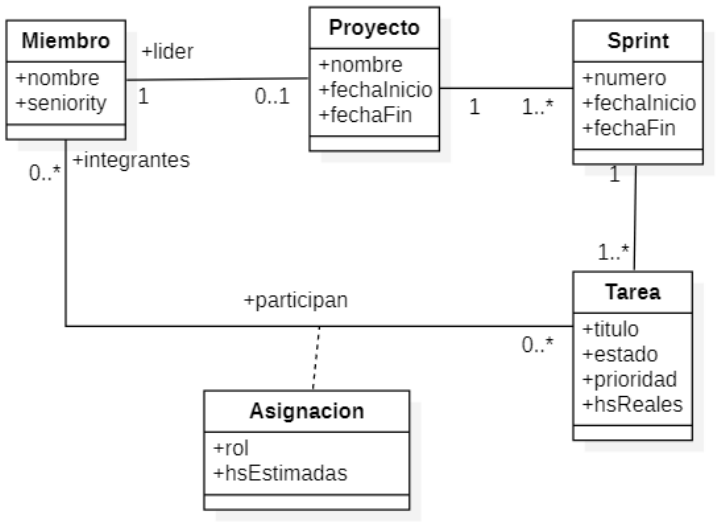
Primer Examen Regular
Análisis y Diseño de Sistemas II
Noviembre 2025

Nombre y Apellido: **Federico Rodriguez**

Dado el siguiente modelo, resolver las operaciones e invariantes en OCL



Especificar las siguientes operaciones:

1. Las tareas cuya prioridad es mayor a 7 y en estado “Finalizada”.
   Context: Tarea :: prioridad() : Set(Tarea)
   Pre:--
   Post: resutl= tarea.allInstances() -> Select(t : Tarea | t.prioridad > 7 and t.estado= “Finalizada”)

2. Los miembros asignados a tareas cuya suma de *horasEstimadas* supera las 40 horas.
   Context: Meimbros :: *horasEstimadas* () : Set(Miembros)
   Pre:--
   Post: resutl= miembros.allInstances() -> Select(m : Miembro | m.Asignacion -> size(a : Asignacion| a.hsEstimadas) >40)

3. Calcular el total de horas estimadas para un Sprint

4. Si existen proyectos que estén en riesgo, es decir que tengan al menos una tarea cuya prioridad sea mayor a 8 y no tenga ningún miembro Senior asignado.
   Context: Proyecto :: riesgo() : Boolean
   Pre:--
   Post: resutl= proyecto.allInstances -> exists(p: Proyecto | p.Miembro -> exists(m: Miembro | m.seniority = true) and p.Miembro.participan -> exists(t : Tarea | t.prioridad = 8)

5. Los miembros que participan en proyectos que tienen tareas finalizadas
   Context: Miembro :: prioridad() : Set(Miembro)
   Pre:--
   Post: resutl= Miembro. allInstances() -> Select(m : Miembro | m.participan -> forAll(t: Tarea | t.estado = “Finalizada”))

Especificar las siguientes invariantes

1. Una tarea en estado “Finalizada” debe tener al menos una asignación y horasReal > 0
   Context: Tarea
   Inv: self.estado= “Finalizada” implies self.Asignacion -> notEmpty() and self.hsReales >0

2. Los miembros integrantes asignados a un proyecto no son líderes del mismo
   Context: Miembro
   Inv: forAll(m.participan -> notEmpty() implies Proyecto.lider -> isEmpty())

3. Un miembro Senior no puede tener asignaciones con rol “Junior”
   Context: Miembro
   Inv: self.seniority = true implies not self.Asignacion.rol= ‘Junior’

4. Todos los sprints de un proyecto deben tener al menos una tarea en estado “En Progreso”
   Context: Sprint
   Inv: forAll(s | s.Proyecto -> notEmpty() implies s.Tarea -> exists(t :tarea | t.estado = “En Progreso”))

5. Todas las tareas de alta prioridad (>7) deben tener al menos un miembro Senior
   Context: Tarea
   Inv. forAll(t | t.prioridad >7 implies t.integrantes -> esxists(m | m.seniority = true))